

Programação de Sistemas Linux

Aula 08 — Dispositivos, /proc e sysfs

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br



Abertura

Notes

Notes

Aula 7 (feito)

- Scripts Bash: variáveis, expansões
- Redirecionamento e *pipes*
- Controle de fluxo, funções
- `getopts` e `trap`
- `grep/sed/awk`

Aula 8 (hoje)

- Dispositivos de **caractere** e de **bloco**
- `/dev` e `udev`
- Controlando dispositivos com `ioctl()`
- Estado do sistema via `/proc` e `/sys`



Objetivos da aula

Ao final desta aula, você será capaz de:

- ✓ Diferenciar dispositivos de **caractere** e de **bloco**, e explicar o papel de *major/minor number*
- ✓ Explicar como `/dev` é populado dinamicamente pelo `udev`
- ✓ Usar `ioctl()` para operações que não são leitura/escrita
- ✓ Ler informações de processos e do kernel em `/proc`
- ✓ Navegar em `/sys` para consultar e configurar dispositivos
- ✓ Saber **quando** usar `/proc` e quando usar `/sys`

Regra de hoje

Depois de hoje, `/dev`, `/proc` e `/sys` deixam de ser pastas misteriosas do `ls /:` são a interface que o kernel expõe para **tudo** — dispositivos, processos, hardware — sem precisar de uma chamada de sistema especial para cada coisa.



Dispositivos Linux

Notes

“No Unix, tudo é um arquivo”

Definição

Um **dispositivo** (*device*) no Linux é representado por um **arquivo especial** em `/dev` – você abre, lê e escreve nele com as mesmas chamadas de sistema (`open`, `read`, `write`, `close`) que usaria em um arquivo comum.

- Isso é o que permite `cat arquivo.txt > /dev/lp0` imprimir um arquivo, ou `cat /dev/urandom | head -c 16` ler bytes aleatórios – **o programa nem precisa saber** que está falando com um dispositivo, não com um arquivo em disco
- Quem faz a “mágica” de traduzir `read/write` em operações de hardware é o **driver** do dispositivo, dentro do kernel

Notes



	Caractere (c)	Bloco (b)
Unidade de E/S	fluxo de bytes, sequencial	blocos de tamanho fixo, endereçáveis
Acesso aleatório	normalmente não	sim (<i>seek</i>)
Passa por cache?	não, geralmente direto	sim, o kernel usa <i>buffer cache</i>
Exemplos	teclado, mouse, terminal, <code>/dev/null</code>	disco (<code>/dev/sda</code>), partição, pendrive

```
1 $ ls -l /dev/sda /dev/tty1
2 brw-rw---- 1 root disk 8,  0 ago 20 10:00 /dev/sda
3 crw--w---- 1 root tty  4,  1 ago 20 10:00 /dev/tty1
```

- A **primeira letra** da listagem (b ou c) já diz o tipo — não é um arquivo comum (-) nem um diretório (d)



Major e minor number

```
1 $ ls -l /dev/sda /dev/sda1 /dev/sda2
2 brw-rw---- 1 root disk 8,  0 ago 20 10:00 /dev/sda
3 brw-rw---- 1 root disk 8,  1 ago 20 10:00 /dev/sda1
4 brw-rw---- 1 root disk 8,  2 ago 20 10:00 /dev/sda2
```

- Onde normalmente apareceria o **tamanho** do arquivo, um dispositivo mostra dois números: **major** e **minor**
- **Major number**: identifica **qual driver** trata esse dispositivo (major 8 = discos SCSI/SATA)
- **Minor number**: identifica **qual instância**, para aquele driver (sda=0, sda1=1, sda2=2...)
- O kernel usa o par (major, minor) para rotear a chamada até a função certa do driver — *não* o nome do arquivo, que é só uma conveniência para humanos

```
1 $ sudo mknod /dev/meudispositivo c 240 0 # cria um no de dispositivo
    manualmente
```



/dev e udev: quem cria esses arquivos?

- Antigamente, /dev era uma pasta **estática**, com um nó para *todo* dispositivo imaginável (milhares de arquivos, a maioria inútil na sua máquina)
- Hoje, /dev é montado como `tmpfs` (em memória) e populado **dinamicamente** pelo **udev**, um daemon que escuta eventos do kernel
- Quando você pluga um pendrive: kernel detecta o hardware → emite um evento pelo `netlink` → `udev` recebe o evento → cria `/dev/sdb1` (e pode disparar regras, como montar automaticamente)

```
1 $ udevadm info --query=all --name=/dev/sda
2 $ udevadm monitor          # observa eventos em tempo real (plugue
                             um pendrive!)
```

Na prática

É por isso que o **nome** de um dispositivo (`/dev/sdb` vs. `/dev/sdc`) pode **mudar** entre reinicializações, dependendo da ordem de detecção — scripts de produção preferem identificar discos por `/dev/disk/by-uuid/` ou `by-id/`, que o `udev` também popula.



Notes

Notes

Além de read/write: ioctl()

O problema

`read()` e `write()` só sabem fazer uma coisa: mover bytes. Mas e se eu quiser saber o **tamanho** de uma partição, **ejetar** um CD-ROM, ou perguntar o **tamanho do terminal** em colunas e linhas? Nenhuma dessas operações é “ler” ou “escrever” dados.

`ioctl()` **resolve isso**: é uma chamada de sistema “genérica” para operações **específicas de cada dispositivo**, que não se encaixam no modelo simples de fluxo de bytes.

```
1 #include <sys/ioctl.h>
2
3 int ioctl(int fd, unsigned long request, ...);
```



Notes

ioctl() na prática: tamanho do terminal

```
1 #include <sys/ioctl.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 int main(void) {
6     struct winsize ws;
7
8     if (ioctl(STDOUT_FILENO, TIOCGWINSZ, &ws) == -1) {
9         perror("ioctl");
10        return 1;
11    }
12
13    printf("Linhas: %d Colunas: %d\n", ws.ws_row, ws.ws_col);
14    return 0;
15 }
```

- `fd`: o descritor do dispositivo (aqui, a saída padrão, que é um terminal)
- `request`: uma constante que diz **o que** pedir — `TIOCGWINSZ` = “Get Window Size”
- Terceiro argumento: um ponteiro para a **estrutura** que o kernel vai preencher (aqui, `struct winsize`) — o tipo dela depende do `request`



Notes

ioctl(): o preço da flexibilidade

- Cada request (TIOCGWINSZ, BLKGETSIZE, CDROMEJECT...) é definido pelo **driver** do dispositivo — não existe uma lista universal, é preciso consultar o cabeçalho/documentação de cada driver
- Por não ter uma assinatura fixa (o terceiro argumento muda de tipo conforme o request), é fácil de usar errado — o compilador **não consegue checar** se você passou a estrutura certa
- Por isso, para as necessidades **mais comuns** (ler informação de dispositivo, estado de processo, parâmetros do kernel), o Linux prefere expor tudo como **arquivos de texto** em `/proc` e `/sys` — muito mais simples de ler de um script, sem precisar compilar nada



/proc: o estado do sistema como arquivos

Notes

Notes

Definição

/proc é um **sistema de arquivos virtual** (*procfs*): seus arquivos não existem em disco — são gerados **sob demanda** pelo kernel, no momento em que você os lê.

```
1 $ ls -la /proc/1234/      # informacoes do processo com PID 1234
2 $ cat /proc/cpuinfo      # informacoes da CPU
3 $ du -sh /proc/cpuinfo   # "tamanho" do arquivo
4 0      /proc/cpuinfo
```

- du mostra **tamanho zero** — porque não há bytes gravados em disco, o conteúdo é **montado na hora** da leitura
- É por isso que cat /proc/uptime sempre mostra um valor **diferente** a cada execução



/proc/[pid]: uma pasta por processo

```
1 $ ls /proc/1234/
2 cmdline cwd environ exe fd maps status stat ...
3
4 $ cat /proc/1234/cmdline | tr '\0' ' ' # comando que iniciou o processo
5 $ cat /proc/1234/status | head -5     # nome, estado, PID, PPID...
6 $ ls -l /proc/1234/fd/                # descritores de arquivo abertos
7 $ ls -l /proc/1234/exe                # link simbolico para o executavel
```

- Comandos como ps, top e lsof **não** têm acesso privilegiado especial — eles só **leem** /proc, exatamente como você acabou de fazer
- /proc/self é um atalho especial: sempre aponta para o próprio processo que está lendo



/proc do sistema (não por processo)

```
1 $ cat /proc/cpuinfo      # modelo, nucleos, flags da CPU
2 $ cat /proc/meminfo      # memoria total, livre, em cache...
3 $ cat /proc/uptime       # tempo ligado (segundos) e tempo ocioso
4 $ cat /proc/loadavg      # carga media do sistema (1, 5, 15 min)
5 $ cat /proc/mounts       # sistemas de arquivo montados
6 $ cat /proc/version      # versao do kernel
```

- É exatamente daqui que ferramentas como free, uptime e mount tiram seus dados
- Alguns arquivos em /proc/sys/ são **graváveis** e ajustam o comportamento do kernel em tempo real (ex.: /proc/sys/net/ipv4/ip_forward)



Notes

Lendo /proc em C

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main(void) {
5     FILE *f = fopen("/proc/meminfo", "r");
6     char linha[256];
7     if (!f) { perror("fopen"); return 1; }
8     while (fgets(linha, sizeof(linha), f)) {
9         if (strncmp(linha, "MemAvailable", 12) == 0) {
10             printf("%s", linha);
11             break;
12         }
13     }
14     fclose(f);
15     return 0;
16 }
```

- **Nenhuma API especial:** é fopen/fgets/fclose, como qualquer arquivo texto
- Esse é o padrão que muitas ferramentas de monitoramento usam para não depender de bibliotecas extras



Notes

sysfs: dispositivos e drivers

Notes

O que é /sys

Definição

sysfs (montado em /sys) é outro sistema de arquivos virtual, que expõe a **árvore de dispositivos e drivers** do kernel — cada dispositivo, barramento e driver vira um diretório com arquivos representando seus atributos.

- Surgiu **depois** do /proc para separar as responsabilidades: /proc ficou focado em **processos e o kernel em geral**; /sys ficou focado em **dispositivos e drivers**, de forma organizada e padronizada
- É a fonte de dados que o udev consulta para criar e nomear os arquivos em /dev

Notes



Navegando em /sys

```
1 $ ls /sys/class/net/           # interfaces de rede
2 eth0  lo  wlan0
3
4 $ cat /sys/class/net/eth0/address # endereço MAC
5 $ cat /sys/class/net/eth0/operstate # "up" ou "down"
6
7 $ ls /sys/block/               # discos e partições
8 sda  sda1  sda2
9
10 $ cat /sys/block/sda/size       # tamanho em setores de 512
    bytes
11 $ cat /sys/block/sda/queue/rotational # 1 = disco rotacional, 0 = SSD
```

- Cada arquivo, em geral, guarda **um único valor** (diferente de /proc/cpuinfo, que mistura tudo num arquivo só)
- Arquivos graváveis permitem **ajustar** o dispositivo (ex.: mudar o brilho de um LED em /sys/class/leds/)



Notes

/proc vs. /sys: quando usar qual

	/proc	/sys
Foco	processos + estado geral do kernel	dispositivos e drivers
Organização	histórica, menos padronizada	um valor por arquivo , padronizada
Uso típico	ps, top, /proc/[pid]/	udev, atributos de hardware
Exemplo	/proc/meminfo	/sys/class/net/eth0/address

Informação sobre um **processo** → /proc.
Informação sobre um **dispositivo** → /sys.



Notes

Encerramento

Notes

Síntese da aula

Conceito	Para que serve
Dispositivo de caractere (c)	fluxo de bytes sequencial (teclado, terminal)
Dispositivo de bloco (b)	blocos endereçáveis, com cache (disco)
<i>Major/minor number</i>	identificam driver e instância do dispositivo
<code>udev</code>	popula <code>/dev</code> dinamicamente, a partir de eventos do kernel
<code>ioctl()</code>	operações específicas de dispositivo, fora de read/write
<code>/proc</code>	processos e estado geral do kernel, como arquivos texto
<code>/sys</code>	dispositivos e drivers, um valor por atributo

A ideia central

Tudo o que hoje vimos – dispositivos, processos, hardware – é acessível com as **mesmas** ferramentas que vocês já dominam: `cat`, `ls`, `fopen`. O kernel prefere expor informação como arquivo sempre que possível, e reserva `ioctl()` para o que realmente não cabe nesse modelo.

Notes



Laboratório 07 — Dispositivos, /proc e sysfs

Ao final deste mesmo encontro, em conjunto com a turma:

- 1 Identificar dispositivos de caractere/bloco e seus *major/minor numbers* em `/dev`
- 2 Consultar atributos de um dispositivo real via `/proc` e `/sys`
- 3 Um pequeno programa em C usando `ioctl()` ou lendo `/proc`



Notes

Próxima aula

Avaliação 1

- Avaliação **prático-teórica**, em laboratório, 2h
- Cobre da **Aula 1 até esta aula (8)**: ferramentas de desenvolvimento, sistema de arquivos, chamadas de sistema, `boot/systemd`, shell scripting, dispositivos/`/proc//sys`
- Bloco de questões conceituais + exercício prático de implementação/depuração, com entrega dos arquivos ao final

Revisão

Vale revisar os guias de atividade e gabaritos dos Laboratórios 01 a 07 antes da avaliação — os exercícios de lá são um bom indicativo do nível de profundidade esperado.



Notes

- HALLINAN, Christopher. *Embedded Linux Primer: a practical real-world approach*. 2. ed. Prentice Hall, 2011. — capítulos sobre dispositivos, `/proc` e `/sys`
- `man 5 proc`, `man 2 ioctl`, `man 7 udev` — referência oficial, sempre a fonte mais confiável e atualizada
- Documentação do kernel em `Documentation/filesystems/sysfs.rst` (código-fonte do kernel Linux)


